SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE – AI / 2024	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

*I **J KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

Q1. Case Study: E-Commerce Product Sorting System (Merge Sort)

An e-commerce platform must sort millions of products based on price and ratings to ensure fast user access. Analyze how Merge Sort can be used to handle large datasets efficiently. Clearly identify key issues such as stability and scalability. Apply divide and conquer concepts to explain the sorting process. Analyze its time complexity and performance. Design a feasible solution and justify why Merge Sort is suitable for this scenario.

Q1. E-Commerce Product Sorting System (Merge Sort)

1. Problem Understanding

An e-commerce platform must sort a catalog running into millions of products by price and by rating so that users experience fast, relevant browsing and filtering. The core challenge is not simply arranging numbers — it is doing so at massive scale while preserving the relative order of products that tie on the sorting key (for example, two products at the same price should keep their original listing order, such as newest-added-first). The sort must also complete within a predictable time budget so catalog refreshes do not degrade the user experience.

2. Key Issues Identified

- **Stability:** products with equal price/rating must retain their relative order (important when a secondary criterion like 'recently listed' matters to sellers and users).
- **Scalability**: the algorithm must handle tens of millions of records without a steep drop in performance.
- **Predictable worst-case behaviour**: unlike data-dependent algorithms, the platform needs guaranteed performance regardless of how the data arrives (already sorted, reverse sorted, or random).
- **External/parallel processing:** catalog data is often too large for a single machine's memory, so the algorithm should be adaptable to external or distributed sorting.

3. Divide and Conquer Approach

Merge Sort follows the classic three-step divide-and-conquer pattern. Divide: the product list of size n is recursively split into two halves until each sub-list contains a single element (trivially sorted). Conquer: each half is sorted independently (recursively applying the same split). Combine: two sorted halves are merged by repeatedly comparing the front elements of each half and appending the smaller (or equal, taken from the left half first to preserve stability) to the output list. This recursive halving guarantees that the recursion tree has exactly log2(n) levels, and every level does O(n) work during the merge step.

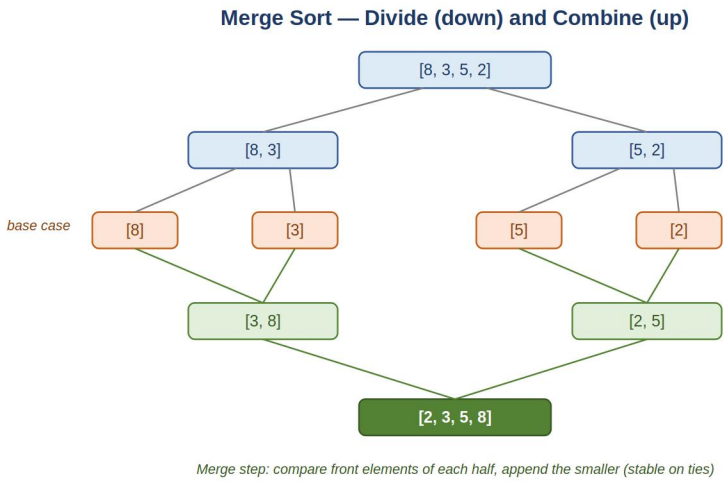

Fig. 1 — Merge Sort recursion tree: divide down to single elements, then merge back up into a sorted array.

4. Complexity and Performance Analysis

Merge Sort has a time complexity of O(n log n) in the best, average, and worst cases, because the divide step always splits the array evenly and the combine step always performs a linear-time merge — there is no data-dependent slowdown as with Quick Sort. Its space complexity is O(n) due to the auxiliary arrays used during merging (or O(n) with in-place variants trading additional time). For an e-commerce catalog of, say, 10 million products, O(n log n) sorting is comfortably fast enough for periodic re-indexing (e.g., nightly batch jobs or after each price update batch), and its predictable performance avoids the risk of an occasional slow run affecting service-level agreements.

5. Proposed Solution and Justification

The recommended design sorts the catalog using Merge Sort as the batch-indexing algorithm feeding a search/index layer (e.g., an inverted index or sorted materialized view) that the live application queries. For very large catalogs, an external Merge Sort variant is used: the catalog is split into chunks that fit in memory, each chunk is sorted independently, and the sorted chunks are merged using a k-way merge, which is a direct extension of Merge Sort's merge step. This design is well suited for multi-server, distributed processing since independent chunks can be sorted in parallel and merged afterward.

Merge Sort is justified over alternatives here because: (a) its stability preserves secondary ordering that matters to business logic (e.g., tie-breaking by listing date), (b) its guaranteed O(n log n) worst case avoids unpredictable slow sorts that could delay catalog updates, and (c) its divide-and-conquer structure naturally parallelizes and extends to external/distributed sorting, which is essential once the dataset no longer fits in a single machine's memory.

Q2. Case Study: Real-Time Ranking System (Quick Sort)

A real-time ranking system updates scores dynamically and needs fast sorting for leaderboard display. Analyze how Quick Sort can be applied in this scenario. Identify issues such as pivot selection and worst-case performance. Apply divide and conquer concepts to explain its working. Analyze its efficiency under different cases. Propose an optimized solution and justify improvements like randomized pivot selection.

Q2. Real-Time Ranking System (Quick Sort)

1. Problem Understanding

A real-time leaderboard must re-order player or item scores as updates arrive, displaying the current ranking with minimal latency. Unlike a one-time batch sort, this system re-sorts frequently, so average-case speed and low memory overhead matter more than stability, since scores rarely have meaningful 'ties that must preserve arrival order' in the same way catalog listings do.

2. Key Issues Identified

- **Pivot selection:** a poor pivot (e.g., always picking the first or last element) degrades performance badly on sorted or adversarial data, which can occur naturally when scores update incrementally near-sorted order.
- **Worst-case time complexity:** naive Quick Sort can hit O(n^2) on unlucky pivot choices, which is unacceptable for a real-time system with strict latency requirements.
- **In-place, low-memory requirement:** frequent re-sorts favor an algorithm that avoids large auxiliary memory allocation.
- **Concurrent updates:** scores can change while a sort is in progress, requiring careful synchronization or snapshotting.

3. Divide and Conquer Approach

Quick Sort divides the array by selecting a pivot element and partitioning the remaining elements into two groups: those less than the pivot and those greater than or equal to it, placing the pivot in its final sorted position during this step. It then conquers by recursively applying the same partitioning process to the two sub-arrays on either side of the pivot. There is no explicit combine step, since the partitioning already leaves the array sorted in place once every element has found its correct position through recursive partitioning.

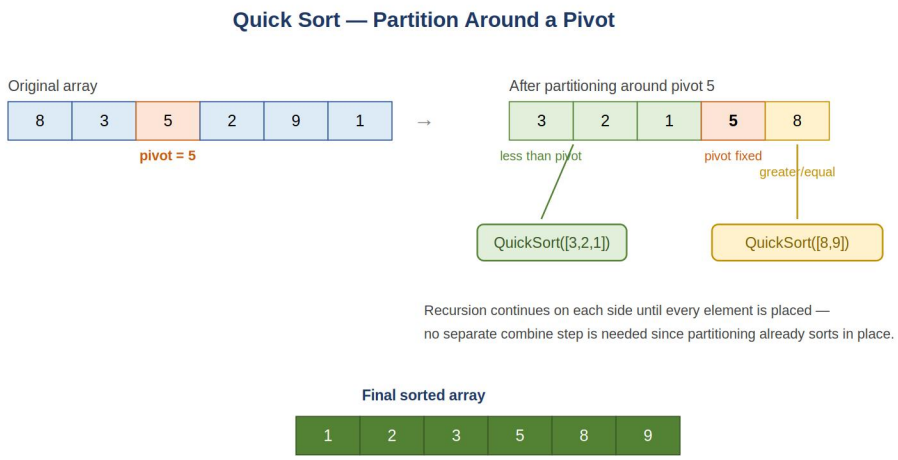

Fig. 2 — Quick Sort partitioning around a pivot, followed by recursive sorting of each side.

4. Complexity and Performance Analysis

With a well-chosen pivot, Quick Sort achieves O(n log n) average-case time and sorts in place with O(log n) auxiliary space for the recursion stack, making it faster in practice than Merge Sort for in-memory data due to better cache locality and lower constant factors. However, with a poor deterministic pivot strategy, the worst case degrades to O(n^2), which is a real risk for a leaderboard where scores may already be nearly sorted between refresh cycles.

5. Proposed Solution and Justification

The recommended design uses Randomized Quick Sort, where the pivot is chosen uniformly at random (or via a median-of-three heuristic comparing the first, middle, and last elements) before each partition step. This makes the worst-case O(n^2) scenario extremely unlikely regardless of the input pattern, since the randomization is independent of how the data is arranged. For incremental updates (a single score changing) rather than a full re-sort, the design should combine Quick Sort for periodic full re-ranks with an incremental re-insertion into a partially sorted structure to avoid re-sorting the entire leaderboard on every single update.

Randomized/median-of-three Quick Sort is justified because it delivers the best practical average-case speed with minimal memory overhead, directly addressing the real-time system's latency and resource constraints, while the randomization specifically neutralizes the adversarial or near-sorted input patterns that would otherwise trigger the O(n^2) worst case.

Q3. Case Study: Search Engine Query Processing (Binary Search)

A search engine maintains a sorted index of keywords and must retrieve results quickly. Analyze how Binary Search improves search efficiency. Identify key requirements such as sorted data and fast retrieval. Apply divide and conquer principles to explain the process. Analyze time complexity and performance. Design a solution and justify why Binary Search is optimal in this context.

Q3. Search Engine Query Processing (Binary Search)

1. Problem Understanding

A search engine keeps a sorted index of keywords (an inverted index) and must resolve user queries into matching documents with very low latency, often within milliseconds, across an index containing millions of terms. The core requirement is fast lookup in already-sorted data, not sorting itself.

2. Key Issues Identified

- **Sorted data precondition:** Binary Search only works correctly on a sorted keyword index, so the index must be kept sorted or re-sorted whenever new terms are added.
- **Fast retrieval under high query volume:** the search must scale to many simultaneous queries with minimal per-query cost.
- **Handling near-matches/prefixes**: pure Binary Search finds exact matches efficiently, but real search engines also need prefix or fuzzy matching, which requires index structures built around the sorted property (e.g., tries or sorted term ranges) rather than Binary Search alone.
- **Index updates:** inserting new keywords into a sorted array is costly, so updates are usually batched rather than applied one at a time.

3. Divide and Conquer Approach

Binary Search repeatedly divides the sorted keyword index in half: it compares the target keyword with the middle element; if they match, the search terminates; if the target is smaller, the search recurses (or iterates) on the left half; if larger, it recurses on the right half. Each comparison eliminates half of the remaining search space, which is the divide-and-conquer principle applied to searching rather than sorting — there is effectively no combine step, since only one half is ever explored.

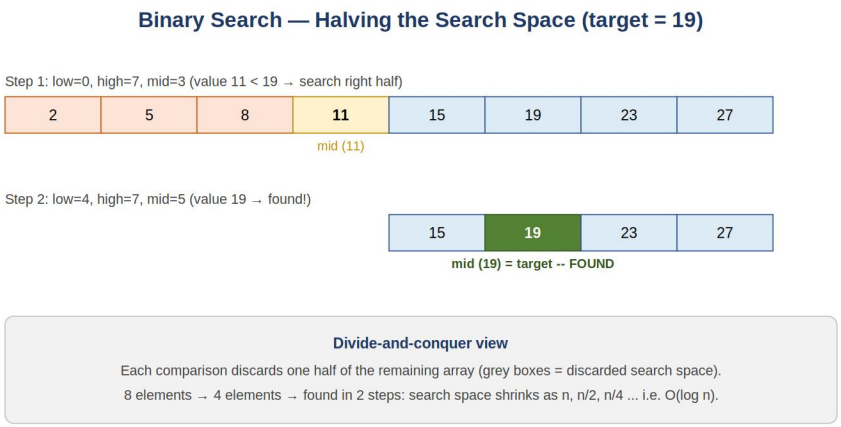

Fig. 3 — Binary Search narrowing the search range by half at each comparison.

4. Complexity and Performance Analysis

Binary Search runs in O(log n) time, since each comparison halves the search space; for an index of even 10 million keywords, this means at most about 24 comparisons to locate any term, which is why it is dramatically faster than a linear O(n) scan through the index. Its space complexity is O(1) for the iterative version (or O(log n) for a recursive version due to call-stack depth).

5. Proposed Solution and Justification

The recommended design maintains the keyword index as a sorted array or sorted array-backed structure (such as a sorted skip list or a balanced tree indexed for range queries), rebuilt or merged in batches as new documents are crawled. Query resolution uses Binary Search to locate the exact keyword entry, which then points to a posting list of matching document IDs. For partial/prefix queries, the same sorted-order property is exploited by binary-searching for the start and end of the matching range.

Binary Search is justified as the optimal choice here because the index is naturally sorted for indexing purposes, query latency requirements are strict, and O(log n) lookup is close to the theoretical best possible for comparison-based search on static or batch-updated sorted data, making it far superior to a linear scan at web scale.

Q4. Case Study: Image Processing System (Matrix Multiplication)

An image processing system performs transformations using large matrices. Analyze how divide and conquer matrix multiplication (e.g., Strassen's method) can improve performance. Identify computational challenges in large matrix operations. Apply the divide and conquer approach to explain optimization. Analyze time complexity and compare with standard multiplication. Design a suitable solution and justify its efficiency.

Q4. Image Processing System (Matrix Multiplication)

1. Problem Understanding

An image processing system represents images and transformation operations (rotation, scaling, filtering, convolution) as matrices, and applies these transformations through matrix multiplication. High-resolution images translate into very large matrices, so the multiplication cost directly determines how fast transformations can be applied, especially for real-time or batch processing of many images.

2. Key Issues Identified

- **Computational cost:** the standard matrix multiplication algorithm is O(n^3) for n x n matrices, which becomes expensive as image resolution (and hence matrix size) grows.
- **Memory bandwidth:** large matrices require significant memory movement, which can bottleneck performance even before arithmetic cost is considered.
- **Numerical precision:** recursive/optimized methods must preserve accuracy for image quality, avoiding compounding rounding errors.
- Scalability to very large or non-square matrices commonly seen in image transformation pipelines.

3. Divide and Conquer Approach

Divide-and-conquer matrix multiplication splits each n x n matrix into four (n/2) x (n/2) sub-matrices. The naive recursive approach still needs 8 sub-multiplications to combine into the result, giving no improvement over the standard method. Strassen's algorithm improves on this by cleverly combining the sub-matrices into 7 (instead of 8) recursive multiplications using a fixed set of additions and subtractions, then recombining the 7 products through further additions/subtractions to reconstruct the four quadrants of the result matrix. This is the divide (split into quadrants), conquer (recursively multiply using only 7 products), and combine (reconstruct the result via additions/subtractions) pattern.

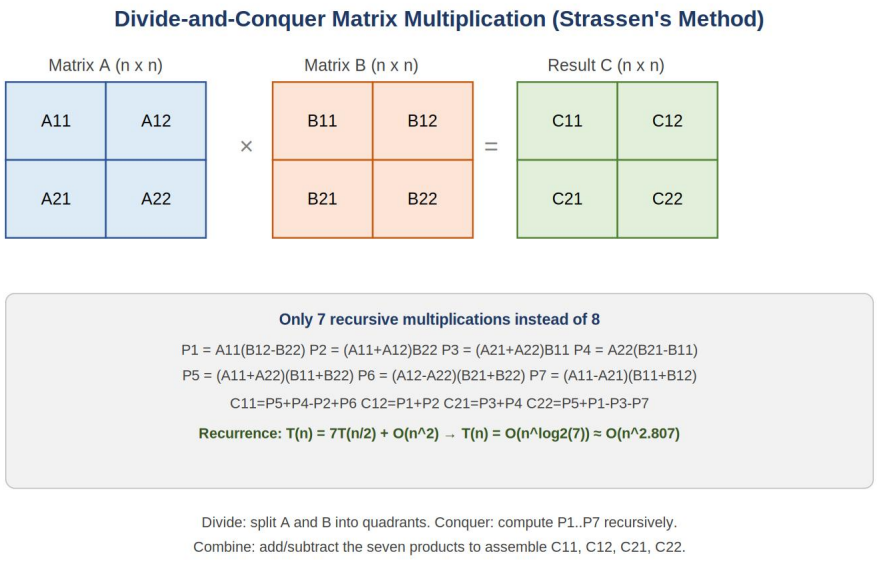

Fig. 4 — Strassen's divide-and-conquer matrix multiplication using 7 sub-products instead of 8.

4. Complexity and Performance Analysis

Standard matrix multiplication runs in O(n^3) time. Strassen's algorithm reduces this to approximately O(n^2.807), because the recurrence T(n) = 7T(n/2) + O(n^2) solves (via the Master Theorem) to n^(log2 7). For large matrices, this asymptotic improvement yields real savings, though Strassen's algorithm has higher constant factors and is more complex to implement, so in practice it is typically applied above a size threshold, falling back to standard multiplication for small sub-matrices to avoid recursion overhead outweighing the benefit.

5. Proposed Solution and Justification

The recommended design applies Strassen's divide-and-conquer multiplication for large transformation matrices (above a tuned size threshold, e.g., 64x64 or larger), switching to standard triple-loop multiplication below that threshold where its lower constant factor wins out. Sub-matrix multiplications at each recursion level are also natural candidates for parallel execution across CPU cores or GPU threads, further improving throughput for batch image processing.

This hybrid design is justified because it captures Strassen's asymptotic advantage (O(n^2.807) versus O(n^3)) where matrix sizes are large enough for the improvement to outweigh its overhead, while avoiding the algorithm's inefficiency on small sub-problems — giving the image processing system both scalability for high-resolution transformations and efficiency for routine small-matrix operations.

Q10. Case Study: Log File Analysis System (Binary Search + Sorting)
A system analyzes large log files by first sorting entries and then searching for specific events. Analyze how sorting (Merge/Quick Sort) and Binary Search can be combined effectively. Identify key issues such as processing time and data size. Apply divide and conquer techniques in both stages. Analyze overall time complexity. Design a solution and justify its efficiency.

Q10. Log File Analysis System (Binary Search + Sorting)

1. Problem Understanding

A log analysis system must first organize large volumes of log entries (potentially gigabytes of data generated continuously) and then support fast repeated lookups for specific events, timestamps, or error codes. The workload therefore has two distinct phases — a one-time (or periodic) sort followed by many repeated searches — so the two phases should be optimized somewhat independently.

2. Key Issues Identified

- **Large data volume:** log files can be enormous, so the sorting phase must scale efficiently and possibly work in external/batched fashion.
- **Sort-then-search pattern:** since searches vastly outnumber sorts (logs are appended and sorted periodically, but searched constantly by analysts/alerts), the upfront sort cost must be amortized across many fast searches.
- **Stability/order preservation:** when multiple log entries share a timestamp, their original append order often needs to be preserved for accurate event sequencing.
- **Freshness:** new log entries keep arriving, so the system must periodically re-sort or merge new data into the already-sorted structure.

3. Divide and Conquer Approach — Sorting Stage

The log entries are sorted using Merge Sort: the log set is recursively divided into halves until single entries remain, each half is conquered by recursive sorting, and sorted halves are combined via a stable merge that compares timestamps and appends the smaller (or earlier-arriving, on ties) entry first. Merge Sort's stability is particularly valuable here so that same-timestamp events keep their logical sequence.

4. Divide and Conquer Approach — Search Stage

Once sorted, Binary Search is used to answer queries: to find an event at or near a given timestamp, the search compares the target with the middle log entry and recurses into the left or right half depending on the comparison, halving the remaining search space at every step until the entry (or its insertion point, for range queries) is found.

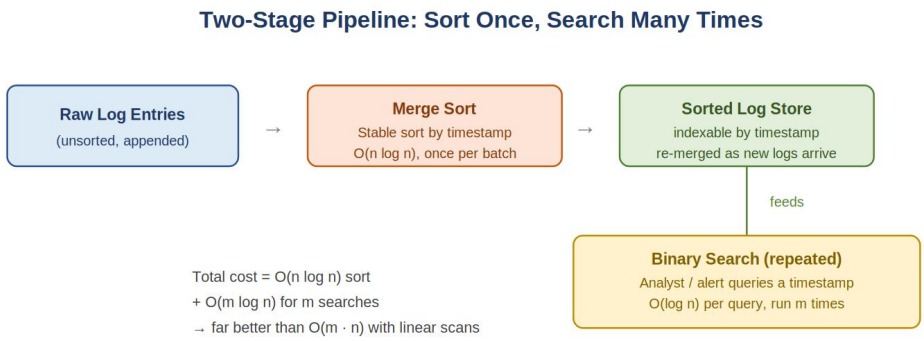

Fig. 5 — Two-stage pipeline: Merge Sort once, then Binary Search repeatedly on the sorted log store.

5. Complexity Analysis

The one-time sorting cost is O(n log n) using Merge Sort. Each subsequent search costs O(log n) using Binary Search. If the system performs m searches against a log set of n entries, the total cost is O(n log n + m log n), which is dramatically better than performing m linear O(n) scans (O(m*n)) once m becomes large — as it does in any real monitoring system that runs continuous or frequent queries.

6. Proposed Solution and Justification

The recommended design maintains logs sorted by timestamp using (external) Merge Sort during ingestion/batch-processing windows, storing the result in a sorted, indexable structure. Analysts and automated monitoring tools then query this structure using Binary Search for exact-timestamp or nearest-timestamp lookups, and range queries are answered by binary-searching for the start and end boundaries of the desired time window. Newly arriving logs are buffered and periodically merged into the main sorted store using the same merge step from Merge Sort, avoiding a full re-sort each time.

This combined design is justified because it matches each algorithm to the phase of the workload it is best suited for: Merge Sort's stable O(n log n) sort handles the bulk, infrequent reorganization of data reliably, while Binary Search's O(log n) lookup efficiently serves the far more frequent search operations — together minimizing total system cost far more effectively than either algorithm could alone.

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____